

Track Copy Mapping Plan

How the funnel and marketing site vary their language between the **clinical** (\$449) and **wellness** (\$149) products, given that a wellness buyer can later upgrade to clinical and redeem the amount already paid.

The short version: one shared lexicon package keyed on a `track` axis, the current track *derived* from succeeded payments rather than stored as a mutable flag, and a test that fails the build if clinical vocabulary appears on a wellness surface.

Why this needs a mapping and not a handful of ternaries

Word choice here is a compliance surface, not only a branding one. `stripe-architecture.md` records that the offering sits at the wellness/medical seam for MCC and HSA/FSA purposes. "Specialist", "diagnosis", "physician", and "treatment" rendered on a wellness-coded purchase are claims about what was sold. Collecting every one of those words into a single reviewable file means compliance sign-off is a file read rather than a grep across two apps.

The copy today also already straddles both registers, on a single product:

File	Register it currently uses
<code>apps/marketing/content/faqs/who-reviews-results.json</code>	Clinical — "our clinical team, including a board-certified neurologist"
<code>apps/marketing/content/faqs/what-do-i-get.json</code>	Wellness — "your Brain Health Navigator"
<code>apps/marketing/content/faqs/navigator-required.json</code>	Wellness — "the navigator conversation"
<code>apps/marketing/content/faqs/medical-care-or-wellness.json</code>	Explicitly both, by design

So part of this work is untangling copy that presently claims both things at once, not just adding a second vocabulary.

The axis: `track`, never price

```
export type Track = "clinical" | "wellness";
```

Prices change; `449` must never appear in a conditional. The Stripe catalog stays the source of truth for amounts (as it is today in `packages/payments/src/pricing.ts`), and `track` is what code branches on.

Add a product catalog beside the existing single-price accessor:

```
// packages/payments/src/catalog.ts
export const PRODUCTS = {
  wellness: { track: "wellness", priceIdEnv: "STRIPE_WELLNESS_PRICE_ID" },
  clinical: { track: "clinical", priceIdEnv: "STRIPE_CLINICAL_PRICE_ID" },
  upgrade: { track: "clinical", priceIdEnv: "STRIPE_CLINICAL_UPGRADE_PRICE_ID" },
} as const;
```

Note `upgrade` carries `track: "clinical"` — it is a *different price* for the *same entitlement*.

Entitlement is derived, not stored

`track` belongs on `payments` as an immutable record of what that purchase was. The user's **current** entitlement is computed:

```
const TRACK_RANK = { wellness: 0, clinical: 1 } as const;
// current track = highest-ranked track among payments where status === 'succeeded'
// no succeeded payments → no entitlement (pre-purchase surfaces)
```

Deriving rather than denormalising onto `users` is deliberate: refunds fall out for free.

`PaymentRefundedEmail` already exists, so refunds are a live path — a refunded clinical upgrade drops the user back to wellness copy automatically, where a mutable `users.current_track` column would leave them reading Specialist language for a product they no longer hold.

Schema delta

Table	Column	Notes
payments	track text not null	'wellness' \ 'clinical' . Written by the Stripe webhook from the price ID. Backfill existing rows to 'wellness' .
linus_enrollments	track text not null	The track the assessment was <i>produced</i> under. See below.

The retroactive-relabel trap

After an upgrade, the assessments dashboard still lists work produced under the wellness track. Re-rendering those cards as "reviewed by your Specialist" is a false statement about work a Navigator actually did — and it is the failure mode most likely to ship unnoticed, because it only appears for upgraded users.

Two different track values coexist on one page:

- **Artifacts** — assessment cards, reports, receipts — render under the track they were produced under. This is why `linus_enrollments` needs its own `track` column and cannot read the user's current entitlement.
- **Chrome and forward-looking copy** — nav, next steps, CTAs — render under current entitlement.

Emails are immutable snapshots and are safe *provided* `track` is passed into the template at send time and never re-derived at render.

The copy package

New workspace package `packages/copy` (`@pbh/copy`). All four consumers — `apps/marketing` , `apps/funnel` , `packages/booking` , `packages/emails` — need it, so it cannot live in an app.

Keys are semantic roles, not words

```
export const LEXICON = {
  clinical: {
    "role.reviewer": { one: "Specialist", many: "Specialists" },
    "role.guide": { one: "Care Coordinator", many: "Care Coordinators" },
    "visit.name": { one: "consultation", many: "consultations" },
  },
  wellness: {
    "role.reviewer": { one: "Brain Health Navigator", many: "Brain Health Navigators" },
    "role.guide": { one: "Brain Health Navigator", many: "Brain Health Navigators" },
    "visit.name": { one: "results review", many: "results reviews" },
  },
} as const satisfies Record<Track, Record<TermKey, Term>>;
```

(Illustrative values — the actual wording is David's call.) The `satisfies` is load-bearing: adding a key to one track and forgetting the other fails typecheck.

Two mechanisms, used deliberately

- **Inline terms** for mentions inside otherwise-identical sentences: `{t("role.reviewer")}`.
- **Whole-phrase keys** wherever the claim itself changes. "Your Specialist will review your results" versus "Your Navigator will walk you through what your results mean" is not a noun swap.

Reach for whole-phrase keys more often than feels necessary. Word-level substitution inside a clinical sentence is how clinical claims leak onto the wellness path.

Upgrade copy needs a third context — but not a third track

A wellness user who can upgrade needs upsell copy neither pure track contains. Model it as track plus flags rather than a `"wellness-upgradable"` track, which would triple the lexicon:

```
type CopyContext = { track: Track; canUpgrade: boolean; creditCents?: number };
```

The `upgrade.*` slice is parameterised (the credit amount is dynamic), so term values must allow functions as well as strings: `t("upgrade.credit", { amount })`.

Enforcement

A test that walks the wellness lexicon plus wellness-track content files and fails on a banned-term regex (`specialist` , `diagnos*` , `physician` , `treatment` , `prescri*` , ...).

It must cover the `upgrade.*` slice under the wellness track. That slice renders on a wellness-coded purchase and its entire purpose is to make the clinical product appealing — it is precisely where someone will reach for "Specialist" to raise conversion. Highest-risk strings in the system.

Threading `track` through

Surface	Where <code>track</code> comes from
Marketing pages	Route / page frontmatter
Booking steps (<code>packages/booking</code>)	Prop, alongside the already-injected per-step server actions
Funnel pre-payment	Route segment (<code>/get-started/clinical</code>) or query param
Funnel post-payment	Derived entitlement
Assessment cards	The row's own <code>linux_enrollments.track</code>
Emails	Explicit argument at send time — these render outside any request context

No module-level global, no implicit context fallback. A missing `track` should be a type error, not a silent default to one of the two products.

Editor layer (Tina)

The lexicon stays in code as the source of truth, mirrored to a `content/global/lexicon/{clinical,wellness}.json` collection so terminology can change ("Navigator" → "Coach") without a deploy.

For pages where the whole narrative differs — pricing, the landing hero — duplicate the page per track rather than tokenising every string. Tokenised narrative copy is unreviewable in the

CMS, which defeats the purpose of having it there.

Redemption mechanism

Assumption being carried: a wellness buyer can upgrade to clinical and redeem what they already paid. Options, in recommended order:

1. **Entitlement-gated upgrade price** (*recommended*) — a separate Stripe Price for the upgrade at the difference, gated server-side on the user having a succeeded wellness payment. Nothing to leak, no coupon arithmetic, and the receipt reads honestly.
2. **Customer-bound promotion code** — if a literal code is a requirement (BHN staff handing it out by phone, a mailer). Stripe promotion codes accept a `customer` restriction plus `max_redemptions: 1` and `expires_at`, so the code is tied to one account rather than bearer-redeemable.
3. **Generic public coupon** — avoid. Ends up on a coupon site.

Whichever is chosen: compute the credit from the actual `payments.amountCents` of the wellness purchase, never a literal `14900`, and clamp with `max(0, clinicalCents - creditedCents)`. If the original purchase was discounted or partially refunded, a hardcoded 149 gives money away.

Delivery

PR	Contents	Status
1	@pbh/copy package: <code>Track</code> , <code>CopyContext</code> , lexicon with only the keys actually needed, resolver, banned-terms test. Convert <code>packages/booking</code> end-to-end to prove the threading.	Done
2	Schema: <code>payments.track</code> , <code>linus_enrollments.track</code> , backfill to 'wellness', derived-entitlement helper, checkout/fulfillment write <code>track</code> from the price.	Done
3	Assessments dashboard: artifact-track vs chrome-track split.	Done
4	<code>packages/emails</code> templates take <code>track</code> as an argument.	Done

PR	Contents	Status
5	Marketing content: Tina lexicon collection, per-track page variants.	Blocked on wording
6	Upgrade flow: upgrade price, entitlement gate, <code>upgrade.*</code> copy slice.	Blocked on credit terms

PRs 1 and 2 are worth doing together even though the upgrade flow is later — the artifact-vs-chrome split is far cheaper to build in now than to retrofit once enrollments exist without a `track` .

What landed, and what it changed

- `packages/copy` — no runtime dependencies, so it is safe in a server component, a client component and a react-email template alike. Nine tests, including one that plants a violation to prove the guard is actually wired to the lexicon rather than merely present.
- `STRIPE_ASSESSMENT_PRICE_ID` keeps its name as the **wellness** price. Only `STRIPE_CLINICAL_PRICE_ID` is new, so no existing environment breaks on deploy.
- Fulfillment resolves the track from PaymentIntent metadata and re-checks the charged amount against *that* track's price. An intent with no `track` is treated as wellness (every payment predating the split was), while a present but unrecognised value is rejected rather than guessed at.
- One live compliance defect was fixed in passing: the booking "Includes" panel promised **"Clinician review of your results"** on the \$149 wellness product. The wellness list no longer makes that claim, and the banned-terms test now fails if it returns.
- Deliberately **not** threaded: `WelcomeEmail` , `AssessmentReadyEmail` , `PaymentReceiptEmail` and `PaymentRefundedEmail` were reviewed and are track-neutral. Only `ReportReadyEmail` says who reviewed the results, so only it takes a `track` .
- Still hardcoded: the `$149` on the booking panel. It is now a prop pinned at the same seam as the track (`BlockRenderer`) instead of buried in the panel, but reading the real amount from the Stripe catalog needs a server boundary this client block doesn't have.

Open decisions

Question	Owner	Blocks
Does the credit expire? Drives a Stripe <code>expires_at</code> and the copy ("redeem within 90 days")	David	PR 6 only
Is the credit the amount actually paid, or a flat \$149?	David	PR 6 only
Actual wording per track — is the wellness-side guide a "Navigator" (matching existing FAQ copy) or a "Coach"?	David	PR 5
Post-upgrade a customer holds two PaymentIntents totalling \$449, potentially under different MCC treatment. Affects the HSA/FSA story and what a full refund means operationally	Bill Laukaitis	Refund runbook, not the build
Confirm the upgrade-with-credit model is real and not just assumed	David	PR 6 only

None of these block PRs 1–4. The copy plumbing is independent of both the credit amount and the final wording.